

KUBEFLOW

Official Documentation

Complete Reference Guide for MLOps & AI Lifecycle

Extracted from www.kubeflow.org/docs | March 2026

Kubeflow is an open-source platform designed to make machine learning workflows on Kubernetes simple, portable, and scalable — from experimentation to production.

Table of Contents

1. Introduction to Kubeflow

- What is Kubeflow?
- Goals & Design Principles
- Kubeflow vs MLflow

2. Architecture Overview

- AI Lifecycle Stages
- Kubeflow Ecosystem Diagram
- Core Components Overview

3. Getting Started

- Prerequisites
- Installing Kubeflow
- Accessing the Dashboard

4. Kubeflow Components

- Kubeflow Pipelines
- Kubeflow Notebooks
- Kubeflow Trainer
- Kubeflow Katib
- KServe (Model Serving)
- Model Registry
- Kubeflow Spark Operator
- Kubeflow Dashboard

5. GenAI with Kubeflow

- GenAI Use Cases
- LLM Fine-Tuning
- Distributed Training

6. Kubeflow Pipelines Deep Dive

- Concepts & Terminology
- Creating Components
- Data Handling
- Control Flow
- Operators Guide

7. Kubeflow Trainer Deep Dive

- PyTorch Training
- JAX Training
- DeepSpeed
- Distributed Cache

8. Katib: Hyperparameter Tuning

- Configuring Experiments
- Algorithms

- Neural Architecture Search

9. Installation & Operations

- Packaged Distributions
- Multi-user Isolation
- Object Store Config

10. External Add-Ons

- Feast (Feature Store)
- Elyra (Pipeline Editor)

11. Community & Resources

- Community & Meetings
- Contributing
- Release History

1. Introduction to Kubeflow

What is Kubeflow?

Kubeflow is an open-source platform that simplifies the deployment and management of machine learning (ML) workflows on Kubernetes. It serves as the foundation for modern AI stacks, providing a unified platform that automates key ML tasks including data preprocessing, model training, hyperparameter tuning, and deployment. At its core, Kubeflow operates as a collection of microservices, making the entire ML lifecycle simple, portable, and scalable.

Kubeflow builds on top of Kubernetes by offering a specialized layer designed specifically for ML workflows using Custom Resource Definitions (CRDs). It uses Kubernetes to manage ML tasks, ensuring they can scale efficiently across multiple nodes through native features like GPU-aware scheduling and StatefulSets for distributed workloads.

Goals & Design Principles

Kubeflow is built on three core principles:

- **Composability** — Modular components that can be mixed and matched to fit your ML stack.
- **Portability** — Run on any cloud provider or on-premises Kubernetes cluster without vendor lock-in.
- **Scalability** — Seamlessly scale from a single laptop to large distributed Kubernetes clusters.

The overarching goal is to make scaling AI models and deploying them to production as simple as possible, letting Kubernetes handle the 'boring stuff' — repeatable deployments, managing loosely-coupled microservices, and scaling based on demand.

Kubeflow vs MLflow

Feature	Kubeflow	MLflow
Primary Focus	End-to-end ML lifecycle on Kubernetes	Experiment tracking & model deployment
Infrastructure	Kubernetes-native, cloud-native	Framework-agnostic, simpler setup
Scale	Large-scale distributed workloads	Lightweight, single or small clusters
Components	Pipelines, Notebooks, Trainer, Katib, KServe, Model Registry	Tracking, Projects, Models, Model Registry
Best For	Production MLOps on cloud/K8s	Experiment tracking, quick prototyping
Cost	Higher (K8s infra required)	Lower (can run locally)

2. Architecture Overview

AI Lifecycle Stages

Developing and deploying an AI application involves multiple iterative stages. Kubeflow provides dedicated components for each phase of the AI lifecycle:

Data Preparation	Ingest raw data, perform feature engineering to extract ML features for the offline feature store, and prepare training data for model development.
Model Development	Choose an ML framework, develop your model architecture, and explore existing pre-trained models for fine-tuning (e.g., BERT or Llama).
Model Training	Train or fine-tune models on large-scale compute environments. Use distributed training when a single GPU cannot handle the model size. Results are stored as model artifacts in the Model Registry.
Model Optimization	Optimize model hyperparameters and apply AutoML algorithms such as neural architecture search and model compression. ML metadata is stored in the Model Registry.
Model Serving	Serve model artifacts for online or batch inference. Models may perform predictive or generative AI tasks depending on the use case.

Core Kubernetes Infrastructure

Kubernetes lies at the heart of Kubeflow's architecture. Key infrastructure components include:

- Dex** — Deployed as a Kubernetes pod; manages user authentication and authorization, enabling secure access to Kubeflow components.
- MinIO** — Provides scalable distributed storage for ML pipeline artifacts. Integrates with Kubeflow Pipelines to store and retrieve data during ML workflows.
- Istio** — Service mesh handling inter-service communication, traffic management, and observability across all Kubeflow microservices.
- Custom Resource Definitions (CRDs)** — Kubeflow uses Kubernetes CRDs to extend the API and define ML-specific resources like TrainJobs, Experiments, and Notebooks.

3. Getting Started

Prerequisites

Before installing Kubeflow, ensure you have:

- A running Kubernetes cluster (v1.27+ recommended)
- kubectl configured and connected to your cluster
- kustomize (v5.0+) for applying Kubeflow manifests
- Sufficient cluster resources (minimum: 4 CPUs, 12 GB RAM for a basic install)
- A default StorageClass configured in your cluster

Installing Kubeflow

Kubeflow can be installed in two primary ways:

1. Packaged Distributions

The Kubeflow AI Reference Platform is available through several packaged distributions that bundle all components into a ready-to-deploy platform. These are maintained by various vendors and community members.

2. Individual Components

You can install standalone Kubeflow components independently without deploying the full platform. This is suitable when you need only specific functionality (e.g., just Kubeflow Pipelines or just Katib).

Quick Install Example (Pipelines standalone):

```
# Install Kubeflow Pipelines standalone
export PIPELINE_VERSION=2.2.0
kubectl apply -k "github.com/kubeflow/pipelines/manifests/kustomize/\
cluster-scoped-resources?ref=$PIPELINE_VERSION"
kubectl wait --for condition=established \
--timeout=60s crd/applications.app.k8s.io
kubectl apply -k "github.com/kubeflow/pipelines/manifests/kustomize/\
env/platform-agnostic-pns?ref=$PIPELINE_VERSION"
```

Accessing the Dashboard

Once installed, access the Kubeflow Central Dashboard by port-forwarding the Istio ingress gateway:

```
kubectl port-forward svc/istio-ingressgateway -n istio-system 8080:80
# Open your browser at: http://localhost:8080
```

Default credentials: user@example.com / 12341234

4. Kubeflow Components

Kubeflow Pipelines

Kubeflow Pipelines (KFP) is the core component for constructing and managing complex ML workflows on Kubernetes. It allows users to create automated, portable, and scalable pipelines as directed acyclic graphs (DAGs). KFP provides a UI for tracking experiments, jobs, and outputs — improving reproducibility and team collaboration. With the KFP SDK, users define ML workflows in Python.

- Define multi-step ML workflows as reusable components
- Pipeline SDK (Python) for programmatic pipeline creation
- Web UI for experiment tracking and artifact visualization
- Support for caching — skip re-running unchanged steps
- IR YAML compilation for portable pipeline definitions
- Multi-user isolation with namespace-scoped experiments

Kubeflow Notebooks

Kubeflow Notebooks provide an interactive development environment running inside Kubernetes pods. Data scientists can launch JupyterLab, VSCode, or RStudio notebooks directly within the cluster, with dynamic resource allocation and integration with other Kubeflow components.

- Launch JupyterLab, VSCode, or RStudio in Kubernetes pods
- Custom container images for specialized ML environments
- Direct access to cluster storage and GPUs
- Integration with Kubeflow Pipelines for notebook-based components
- Resource quotas and namespace isolation per user

Kubeflow Trainer

Kubeflow Trainer (formerly Training Operator) supports scalable distributed model training and LLM fine-tuning. It provides Python APIs and Training Runtimes for seamlessly scaling from single-machine to large distributed clusters. Powered by Apache Arrow and Apache DataFusion, Kubeflow Trainer streams tensors directly to GPU nodes via a distributed cache layer — enabling seamless access to large datasets while minimizing I/O overhead.

- PyTorch, JAX, DeepSpeed, and MLX training support
- TorchTune BuiltinTrainer for LLM fine-tuning
- Distributed Data Cache powered by Apache Arrow

- Local execution mode (process, Docker, Podman backends)
- Job scheduling with Coscheduling, Volcano, and Kueue
- MPI, TensorFlow, PaddlePaddle, XGBoost job support (legacy v1)

Kubeflow Katib

Katib is Kubeflow's AutoML component for hyperparameter optimization and neural architecture search (NAS). It supports multiple search algorithms and integrates with any ML framework through a flexible metrics collection system.

- Bayesian optimization, random search, grid search, CMA-ES
- Neural Architecture Search (DARTS, ENAS, BOHB)
- LLM hyperparameter optimization support
- Early stopping strategies (Median, Percentile)
- Katib UI for experiment visualization
- Custom metrics collectors (StdOut, File, Prometheus, custom)

KServe (Model Serving)

KServe (previously KFServing) is Kubeflow's model serving component. It provides serverless inferencing on Kubernetes with support for major ML frameworks, enabling scalable and production-grade model deployment.

- Serverless inference with autoscaling to zero
- Support for TensorFlow, PyTorch, ONNX, XGBoost, scikit-learn
- Canary rollouts and A/B testing
- Transformer and explainer components
- Multi-model serving for resource efficiency

Model Registry

The Kubeflow Model Registry provides a centralized store for managing ML models throughout their lifecycle — from training artifacts to production deployments. It offers REST and Python client APIs for integration.

- Register and version ML models and artifacts
- Track model metadata and lineage
- REST API (v1alpha3) and Python client
- Model Catalog for sharing models across teams
- Integration with Kubeflow Trainer and KServe

Kubeflow Spark Operator

The Kubeflow Spark Operator enables running Apache Spark workloads natively on Kubernetes. It manages the lifecycle of Spark applications through Kubernetes custom resources.

- SparkApplication and ScheduledSparkApplication CRDs
- Integration with Google Cloud Storage and BigQuery
- Volcano and YuniKorn batch scheduling support
- Integration with Kubeflow Notebooks
- Prometheus/JMX monitoring support
- Leader election and resource quota enforcement

5. GenAI with Kubeflow

Kubeflow is built for modern GenAI workflows, providing first-class support for LLM fine-tuning, large-scale distributed training, and generative model serving.

LLM Fine-Tuning Workflow

- **Step 1 — Data Preparation:** Load and preprocess your dataset. Use Kubeflow Pipelines to orchestrate data ingestion and tokenization steps.
- **Step 2 — Model Selection:** Choose a base model (e.g., Llama, BERT, Mistral) from Hugging Face or your Model Registry.
- **Step 3 — Hyperparameter Optimization:** Use Katib to optimize learning rate, batch size, LoRA rank, and other fine-tuning parameters.
- **Step 4 — Distributed Training:** Launch a TrainJob with Kubeflow Trainer. Use DeepSpeed or FSDP for large models that don't fit on a single GPU.
- **Step 5 — Model Registry:** Store the fine-tuned model artifact and its metadata in the Model Registry.
- **Step 6 — Serving:** Deploy the model via KServe for scalable online inference.

Distributed Training for LLMs

Kubeflow Trainer supports multiple distributed training strategies essential for LLMs:

Strategy	Use Case	Kubeflow Support
Data Parallelism	Same model, different data batches across GPUs	PyTorchJob, TFJob
Model Parallelism	Model too large for single GPU	DeepSpeed, FSDP
Pipeline Parallelism	Split model layers across devices	DeepSpeed
Tensor Parallelism	Distribute individual layers	DeepSpeed, Megatron
LoRA Fine-tuning	Parameter-efficient fine-tuning	TorchTune BuiltinTrainer

6. Kubeflow Pipelines Deep Dive

Key Concepts

Pipeline: A directed acyclic graph (DAG) of components representing an end-to-end ML workflow.

Component: A self-contained step in a pipeline. Can be a Python function, container, or notebook.

Experiment: A logical grouping of pipeline runs for comparison and tracking.

Run: A single execution of a pipeline with specific parameters.

Artifact: Data produced by a component (model, dataset, metrics, etc.) tracked by ML Metadata.

IR YAML: Intermediate Representation — the compiled, portable format for sharing pipelines.

Pipeline Root: The remote storage location (GCS, S3, MinIO) where artifacts are stored.

Recurring Run: A scheduled pipeline run triggered at regular intervals.

Creating a Lightweight Python Component

```
from kfp import dsl
from kfp.client import Client

@dsl.component(base_image='python:3.11')
def preprocess_data(input_path: str, output_path: dsl.OutputPath(str)):
    import pandas as pd
    df = pd.read_csv(input_path)
    # Preprocessing logic here
    df.to_csv(output_path, index=False)

@dsl.component(base_image='python:3.11',
               packages_to_install=['scikit-learn'])
def train_model(data_path: str,
                model_output: dsl.Output[dsl.Model]):
    from sklearn.ensemble import RandomForestClassifier
    import pickle, pandas as pd
    df = pd.read_csv(data_path)
    X, y = df.drop('target', axis=1), df['target']
    model = RandomForestClassifier(n_estimators=100)
    model.fit(X, y)
    with open(model_output.path, 'wb') as f:
        pickle.dump(model, f)
```

```

@dsl.pipeline(name='ml-training-pipeline')
def ml_pipeline(data_path: str = 'gs://my-bucket/data.csv'):
    preprocess_task = preprocess_data(input_path=data_path)
    train_task = train_model(
        data_path=preprocess_task.outputs['output_path'])

# Compile and run
from kfp import compiler
compiler.Compiler().compile(ml_pipeline, 'pipeline.yaml')

client = Client(host='http://localhost:8080')
client.create_run_from_pipeline_func(ml_pipeline, arguments={})

```

Control Flow Patterns

- **Conditions (if/else):** Use `dsl.If()`, `dsl.Elif()`, `dsl.Else()` for conditional branching.
- **Loops:** Use `dsl.ParallelFor()` to iterate over a list of inputs in parallel.
- **Exit Handling:** Use `dsl.ExitHandler()` to run cleanup steps regardless of pipeline outcome.
- **Caching:** Enable caching to skip re-executing components with the same inputs.

7. Kubeflow Trainer Deep Dive

Training a PyTorch Model

```
from kubeflow.trainer import TrainerClient

# Create a TrainJob for distributed PyTorch training
client = TrainerClient()

client.train(
    trainer=client.pytorch(
        func=training_function, # Your training function
        num_workers=4,
        resources_per_worker={
            "gpu": 1,
            "cpu": 4,
            "memory": "16Gi"
        }
    ),
    name="llm-finetune-job"
)

# Monitor job status
job = client.get_job(name="llm-finetune-job")
print(f"Status: {job.status}")
print(f"Conditions: {job.conditions}")
```

Supported Training Frameworks

Framework	Custom Resource	Use Case
PyTorch	TrainJob (v2) / PyTorchJob (v1)	Deep learning, LLM fine-tuning
TensorFlow	TFJob	TF models, Keras workflows
JAX	TrainJob / JAXJob	Research, TPU training
DeepSpeed	TrainJob + DeepSpeed runtime	Very large model training
MLX	TrainJob + MLX runtime	Apple Silicon training

PaddlePaddle	PaddleJob	PaddlePaddle models
XGBoost	XGBoostJob	Gradient boosting
MPI	MPIJob	MPI-based distributed training

8. Katib: Hyperparameter Tuning

Katib is Kubeflow's AutoML component for hyperparameter optimization and neural architecture search. It is framework-agnostic and integrates with any training code through a simple metrics interface.

Supported Search Algorithms

Algorithm	Type	Best For
Random Search	Exploration	Fast baseline, large search spaces
Grid Search	Exhaustive	Small, discrete parameter spaces
Bayesian Optimization	Model-based	Expensive evaluations, few parameters
CMA-ES	Evolutionary	Continuous parameters, complex landscapes
HYPERBAND	Early stopping	Large number of configurations
BOHB	Bayesian+Bandit	Combining Bayesian opt. with early stopping
DARTS	NAS	Differentiable architecture search
ENAS	NAS	Efficient neural architecture search

Example Katib Experiment (Python SDK)

```
from kubeflow.katib import KatibClient
from kubernetes.client import V1ObjectMeta
from kubeflow.katib import V1beta1Experiment, V1beta1ExperimentSpec
from kubeflow.katib import V1beta1ObjectiveSpec, V1beta1ParameterSpec

experiment = V1beta1Experiment(
    metadata=V1ObjectMeta(name="hp-tuning-example"),
    spec=V1beta1ExperimentSpec(
        objective=V1beta1ObjectiveSpec(
            type="maximize",
            goal=0.95,
            objective_metric_name="accuracy"
        ),
        algorithm={"algorithmName": "bayesianoptimization"},
        parameters=[
```

```
V1beta1ParameterSpec(  
  name="learning_rate", parameter_type="double",  
  feasible_space={"min": "0.001", "max": "0.1"}  
)  
V1beta1ParameterSpec(  
  name="batch_size", parameter_type="int",  
  feasible_space={"min": "16", "max": "128"}  
)  
],  
max_trial_count=20,  
parallel_trial_count=3,  
)  
)  
  
client = KatibClient()  
client.create_experiment(experiment)
```

9. Installation & Operations

Packaged Distributions

The Kubeflow AI Reference Platform is available through distributions maintained by vendors and the community. These bundle all components into a unified, ready-to-deploy package:

- Canonical Charmed Kubeflow
- Arrikto Enterprise Kubeflow
- Amazon SageMaker Pipelines (AWS)
- Google Cloud Vertex AI Pipelines
- Azure Machine Learning
- IBM Cloud Pak for Data
- Kubeflow on OpenShift

Multi-user Isolation

Kubeflow supports multi-tenancy through Profiles and Namespaces. Each user gets a dedicated Kubernetes namespace, and RBAC policies control access to resources. Kubeflow Pipelines uses the same namespace isolation to keep experiments and runs separate per team.

Object Store Configuration

Kubeflow Pipelines requires an S3-compatible object store (MinIO, AWS S3, GCS) for storing pipeline artifacts. Configure the pipeline root in your environment:

```
# Set pipeline root to your S3/GCS bucket
import kfp
client = kfp.Client()

# When compiling, specify the pipeline root:
@dsl.pipeline(
    name='my-pipeline',
    pipeline_root='gs://my-bucket/kubeflow/artifacts'
)
def my_pipeline():
    ...
```

10. External Add-Ons

Feast — Feature Store

Feast is an open-source feature store that integrates with Kubeflow. It provides a centralized registry for ML features, enabling consistent feature computation between training and serving — solving the training-serving skew problem.

- Centralized feature registry for feature discovery and documentation
- Offline feature store for training data retrieval
- Online feature store for low-latency serving
- Point-in-time correct feature retrieval to prevent data leakage
- Works with BigQuery, Redshift, Spark, and more

Elyra — Pipeline Editor

Elyra is a set of AI-centric extensions for JupyterLab that integrates with Kubeflow Pipelines. It provides a visual pipeline editor and notebook scheduler, allowing data scientists to build pipelines from notebooks without writing pipeline code.

- Visual Pipeline Editor for drag-and-drop pipeline construction
- Notebook scheduler for converting notebooks to pipeline steps
- Code Snippets browser for reusing common code patterns
- Integration with Kubeflow Pipelines and Apache Airflow
- Git integration for version control within JupyterLab

11. Community & Resources

Community Links

Resource	URL
Documentation	https://www.kubeflow.org/docs/
GitHub	https://github.com/kubeflow/
Blog	https://blog.kubeflow.org/
Slack	https://kubeflow.slack.com
Mailing List	https://groups.google.com/g/kubeflow-discuss
KFP SDK Docs	https://sdk.kubeflow.org/en/latest/
KServe Website	https://kserve.github.io/website/
Feast Docs	https://docs.feast.dev/

Release History (Recent)

Version	Key Features
v1.11 (Latest)	Kubeflow Trainer v2, improved GenAI support, Model Catalog API
v1.10	Kubeflow SDK, Spark Operator improvements, Trainer distributed cache
v1.9	KServe integration improvements, Katib LLM optimization
v1.8	Pipeline v2 GA, multi-arch container images
v1.7	Training Operator v1, Model Registry alpha
v1.6	KFP v2 beta, Notebooks improvements